

**SECJ1023-03
PROGRAMMING TECHNIQUE II**

SEMESTER 2, SESSION 2025/2026

PROJECT 2 REPORT

Food Ordering and Delivery Tracking System

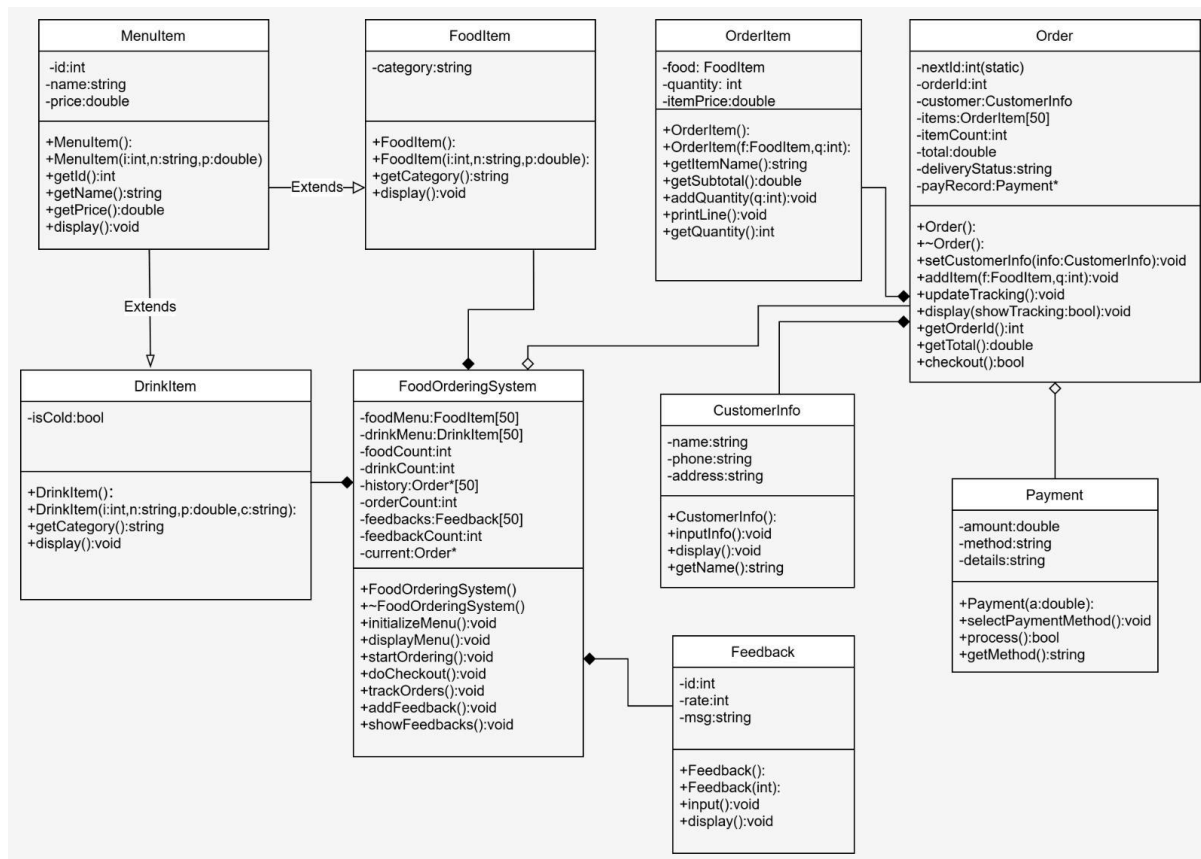
LECTURER: LIZAWATI BINTI MI YUSUF

GROUP NAME: Ctrl C+Ctrl V

GROUP MEMBERS:

			
CHENG ZHI MIN A25CS0050	LIM LI JING A25CS0248	LIM LI NING A25CS0249	NG XUAN YEE A25CS0291

a. UML Class Diagram



b. Class Descriptions

Class	Description
MenuItem	MenuItem is the base class that defines common attributes of all items in the menu. This class not only holds the values for the attributes but also provides accessor methods for each attribute. Besides, the display() method prints out the value of each attribute in a well-aligned manner. The class acts as the parent class for FoodItem and DrinkItem, thus eliminating duplicate code.
FoodItem	FoodItem class is a specialized component of the menu system that represents solid food products. It inherits core attributes like a unique ID,name and price from the MenuItem base class through an inheritance relationship. It extends functionality by adding a specific category string. Its primary role is to specialize the menu structure, enabling the system to effectively categorize and display food items distinctly from beverages for the user.
DrinkItem	DrinkItem class represents beverages.inherits core attributes from

	MenuItem and includes a specific isCold boolean attribute to distinguish drink types for the user.
OrderItem	OrderItem represents a specific line item within a customer's shopping cart. Its main responsibilities include linking a MenuItem to a specific quantity and calculating the subtotal for that particular entry. In the overall system, it acts as a fundamental building block for the Order class through a composition relationship, ensuring that item-specific quantities and costs are tracked accurately throughout the transaction.
CustomerInfo	CustomerInfo is dedicated to managing the personal and contact details of the user placing an order. Its main responsibilities include capturing and storing essential information such as the customer's name, phone number and physical delivery address. Within the system, it composes part of the Order class to ensure that every transaction is tied to specific recipient and location, which is critical for successful delivery.
Payment	It acts as a transaction of payment for the fulfilled order, holding information regarding the total to be paid, the mode of payment selected, and the related information. In addition, it requests the user to choose a payment mode, takes in the required information, and then processes the transaction. This class object is allocated memory on the heap through the Order class at checkout, after which it is deleted through Order's destructor.
Order	Order is made up of a customer's transaction containing information about the ordered items, shipping details, payment, and shipping status. This class aggregates and combines instances of the OrderItem class, assigns an instance of the CustomerInfo class, creates a heap-allocated instance of the Payment class, and keeps track of the shipping status. It connects items, customer details, and payment into one complete transaction record.
Feedback	Feedback is the post-order review submitted by customer that consists of the star ratings and comments along with the order ID of a complete order. The primary functions of this class include collecting star ratings and comments from customers and displaying the review and exposing the order ID linked to it. This class is stored in FoodOrderingSystem's feedback array after checkout.
FoodOrderingSystem	The FoodOrderingSystem class represents the controller of the

	entire system. It is the main class which initiates the menu items into two different arrays (foodMenu and drinkMenu). The FoodOrderingSystem controls all orders placed and completed orders recorded. This class also handles all functionalities such as ordering food, checking out, tracking orders, and giving feedback.
--	--

c. Relationship Explanation

i. Inheritance

MenuItem serves as the base class, while FoodItem and DrinkItem are the derived classes. This relationship is appropriate because all menu products share core attributes such as id, name, and price, which are defined once in the base class to promote code reusability. These attributes are then specialized in the derived classes to meet specific requirements. For instance, FoodItem specializes the base class by adding a category string to classify solid food. Besides that, DrinkItem specializes the base class by adding an isCold boolean to distinguish beverage temperatures. This design allows the system to treat different types of menu items polymorphically through the MenuItem interface, ensuring that the overall system remains modular and easy to extend.

ii. Association

(a) Composition

Order and CustomerInfo	Order and CustomerInfo have a composition relationship because the customer's delivery details are an intrinsic part of a specific transaction. If an Order is deleted, the CustomerInfo does not still exist as it is tied to that specific order's lifecycle, which confirms it is composition.
Order and OrderItem	Order and CustomerInfo have a composition relationship because the customer's delivery details are an intrinsic part of a specific transaction. If an Order is deleted, the CustomerInfo does not still exist as it is tied to that specific order's lifecycle, which confirms it is composition.

FoodOrderingSystem and FoodItem	FoodOrderingSystem and FoodItem have a composition relationship because the food menu is a core internal component of the system's database. If the FoodOrderingSystem is deleted, the FoodItem menu array does not still exist, which confirms it is composition.
FoodOrderingSystem and Feedback	FoodOrderingSystem and Feedback have a composition relationship because the feedback data is stored within the controller's memory space as a core part of its internal state. If the FoodOrderingSystem is deleted, the Feedback entries do not still exist, which confirms it is composition.
FoodOrderingSystem And DrinkItem	FoodOrderingSystem and DrinkItem have a composition relationship because the beverage records are managed and owned entirely by the main controller. If the FoodOrderingSystem is deleted, the DrinkItem array does not still exist, which confirms it is composition.

(b) Aggregation

Order and Payment	Order and Payment have an aggregation relationship because the payment process is a separate functional entity responsible for transaction validation. If the Order is deleted, the Payment logic and its class structure do still exist independently and in some system designs, the payment record might be archived separately, which confirms it is aggregation.
FoodOrderingSystem and Order	FoodOrderingSystem and Order have an aggregation relationship because the system acts as a collection manager for historical records. If the FoodOrderingSystem is deleted, the Order objects stored in the history do still exist as independent entities until specifically deallocated, which confirms it is aggregation.

d. Design Rationale

The overall design of the Food Ordering and Delivery Tracking System is built on Object-Oriented Programming (OOP) principles to ensure the system is structured, logical, and easy to expand.

i. Class Creation Rationale

Each class was created to handle a specific, isolated responsibility to avoid system complexity. The MenuItem base class and its derived classes (FoodItem, DrinkItem) were established to handle the product hierarchy and common attributes like ID and price. OrderItem acts as a bridge to track quantities without modifying the original menu. Furthermore, CustomerInfo and Payment were separated into distinct classes to isolate logistics and financial data, ensuring that the main Order class remains clean and focused on the transaction itself.

ii. Choice of Specific Relationships

The relationships were chosen based on the physical and logical lifecycle of the objects within the system. Inheritance was selected for menu items to promote code reusability and allow polymorphic behavior. Composition was used for Order with CustomerInfo and OrderItem because these parts are essential to the transaction; if an order is deleted, these details should not exist independently. In contrast, Aggregation was chosen for Order and Payment (via pointers) and for the FoodOrderingSystem history to allow for flexible memory management and to signify that these are separate functional entities that can logically exist even if the parent container is modified.

iii. Improvements in Modularity, Maintainability, and Reusability

This design improves software quality by ensuring a high degree of modularity, as each class has a single responsibility and can be tested independently. Maintainability is enhanced because bugs can be isolated within specific classes, such as fixing payment logic without affecting the menu. Finally, the design maximizes reusability through the inheritance structure; for example, new product categories like "Desserts" can be added by simply deriving a new class from MenuItem without rewriting the existing ordering logic.

e. Design Assumptions

i. System Scope Assumptions

NO	Assumption	Rationale
1	Single restaurant only	The menu comes from one restaurant. Multi-vendor support is intentionally out of scope to keep the design simple and focused on ordering workflow.
2	No user authentication	No login system. Customers enter their name, phone, and address directly when ordering. This simplifies the design since user account management is not required.
3	Console application only	No graphical user interface (GUI). The system runs in a terminal/command prompt. This is appropriate for a programming techniques course.

ii. Technical Assumptions

4	Fixed array sizes	Maximum 50 menu items, 50 orders in history, 50 items per order, 50 feedback entries. These limits are sufficient for demonstration and prevent dynamic memory complexity.
5	Simplified payment processing	Payment is simulated. No real API integration with banks, credit card processors, or e-wallet providers. Card details are collected but not actually processed.
6	Manual status update	Delivery status is updated manually by the user for demonstration. No GPS, real-time tracking, or automatic status changes.
7	English only interface	All user interface text, prompts, and output are in English. No multi-language support.

iii. Data Assumption

9	Valid customer input	Basic input validation exists such as quantity > 0, but complex validation phone number format, address verification is not implemented.
---	----------------------	--

10	Static menu	No real-time stock tracking or automatic updates from the kitchen.
11	No file loading on startup	The menu is initialized in code, not loaded from an external file. This simplifies the design but makes menu changes require recompilation.

iv. Business Logic Assumptions

13	No discount or promotions	All items are sold at full price. No promo codes or discount are available
14	No order cancellation after checkout	Once an order is placed, it cannot be cancelled.
15	No delivery fee calculation	Delivery fee is already included in the price of the food.
16	No tax calculation	Tax also already included in the price of the food
17	No removing of order after adding	Items cannot be removed after being added to the order
18	No real rider assignment	Riders are not modeled as objects. The deliveryStatus string simply changes text — no actual rider is assigned to the order.
19	No estimated delivery time	The system does not calculate or display estimated delivery time based on distance, traffic, or preparation time.

v. Security Assumption

20	No password protection	Customer information such as name, phone, address is not protected.
21	No CVV verification	The 3-digit CVV code is collected but NOT verified against any real database. Any 3 digits such as 000, 123, 999 are accepted.

vi. Feedback Assumptions

22	No rating average calculation	The system does not automatically calculate the average rating of all feedbacks.
23	Comment length not limited	The msg string can be arbitrarily long. No limit is enforced.

f. Design Limitations

i. Fixed-size arrays

All collections use statically allocated arrays with hardcoded limits. Memory is wasted when usage is low, and entries are silently dropped when limits are reached.

ii. Feedback limited to latest order

`addFeedback()` only targets `history[orderCount - 1]`, preventing users from reviewing any earlier order in the same session.

iii. Minimal input validation

Outside of quantity checks, user input in areas like `CustomerInfo` is accepted as-is with no format checking, risking empty or corrupt records.

Video Link:

<https://youtu.be/dZzNoVWanJA>